Building a CNN for Rock-Paper-Scissors Classification

Lorenzo Pacacussi

lorenzopacacussi@studenti.unimi.it
State university of Milan, Milan, Italy

Abstract. This project aims to build and compare three different convolutional neural networks (CNNs) for classifying images of hand gestures representing rock, paper, scissors. The three CNNs represent three increasing levels of complexity and were trained on the same standardized dataset containing 2188 images of hand gestures. The training was performed using a train-test split of 70% and 30% (which was then split into a validation set and a test set). Testing was performed using both the test set from the standardized dataset and two custom datasets containing 70 and 40 images respectively, which were collected by the author and contain more challenging images (e.g. with different backgrounds, lighting conditions, hand orientations, etc.). Results show that the most complete CNN achieved the best performance on the standardized test set, with higher validation accuracy than training accuracy, which suggests that the model is not overfitting and is able to generalize well to unseen data. However, the performance on the custom datasets was significantly lower, especially on the second custom dataset which contains images with different backgrounds where the model's performance is similar to a random classifier. Analyzing the results, it can be observed that the model is able to correctly classify images with simple backgrounds and good lighting conditions, but struggles with images that have more complex backgrounds or poor lighting conditions. Another critical aspect is the position of the fingers; error analysis revealed that most of the misclassifications contain finger positions that are similar to other classes.

1 Dataset Exploration

The main dataset used for this project is the Kaggle Rock-Paper-Scissors dataset [1], which contains 2,188 images of hand gestures representing rock, paper, and scissors. The dataset is organized into three folders, one for each class, and each folder contains images with similar orientations and lighting conditions. The images are provided in JPEG format with a resolution of 300×200 pixels.

In addition to the main dataset, two secondary custom datasets were collected[1] by the author, containing 40 and 70 images respectively. These datasets

[1] For privacy reasons, these two datasets will not be displayed in the paper or in the project but performance results will be provided.

were used to evaluate the performance of the CNNs on more challenging and diverse images. The first custom dataset includes hand gesture images captured under different lighting conditions and with varying rotations, while still maintaining a homogeneous background. The second custom dataset contains images with more heterogeneous backgrounds with grid-like patterns, different camera angles, and greater variability overall. Both custom datasets are balanced, with approximately the same number of images for each class.

All datasets contain the same three classes: rock, paper, and scissors. However, the primary training dataset is highly homogeneous, as most images share similar orientations, poses, lighting conditions, and backgrounds. This homogeneity is reflected in the high performance achieved by the model on the training data. The same limitation also affects the validation and test sets, since they are generated by splitting the same original dataset used for training.

Fig. 1: Sample images from the training dataset.

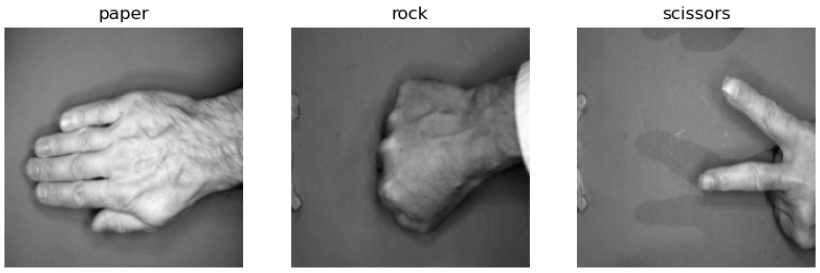
2 Methodology

2.1 Preprocessing

To reduce computational cost and speed up both training and testing, all images in the dataset were resized to 200×200 pixels and converted to greyscale. Converting the images from RGB to greyscale reduces the input channels from three to one, decreasing the amount of data processed by the model.

Additionally, using greyscale images prevents the model from learning color-based features. In this specific application, this is beneficial because the classification task is primarily shape-based. Color information should not significantly influence the classification process and could instead become a source of overfitting, especially considering the high homogeneity of the training dataset, where most images share the same green background.

Images were then normalized by dividing their pixel values by 255 to scale them to the range [0, 1]. This normalization step helps to improve the convergence of the CNN during training by ensuring that the input data is on a consistent scale. This step was implemented in the code as a layer within the model, but conceptually it is a preprocessing step that is applied to the input data before it is fed into the CNN.



The main dataset was split into a training set (70%), a validation set (15%), and a test set (15%). The training set was used to train the CNNs, while the validation set was used to tune hyperparameters and prevent overfitting. The test set was reserved for evaluating the final performance of the models after training.

Data augmentation techniques were applied in the two most complex CNNs to prevent overfitting and improve generalization performance:

- Random rotation: Images were randomly rotated within a range of -36 to 36 degrees since the training data contained images with the same orientation, this was aimed at making the model able to recognize the actual hand gesture and not just the orientation of the fingers.
- Random zoom: Images were randomly zoomed in or out by up to 10% to help the model learn to recognize hand gestures at different distances from the camera and indirectly of different hand sizes.
- Random horizontal flip: Images were randomly flipped horizontally, this was important since the whole dataset contained images with the hand coming into frame from the right side.
- Random brightness adjustment: The brightness of the images was randomly adjusted by up to 40% to make the model more robust to varying lighting conditions.
- Random contrast adjustment: The contrast of the images was randomly adjusted by up to 40% to help the model learn to recognize hand gestures under different lighting conditions, this is not necessary for this dataset but this implementation needs to be robust enough to be usable in a less controlled environment.
- Random translation: Images were randomly translated horizontally and vertically by up to 10% to help the model learn to recognize hand gestures that are not perfectly centered in the frame.

These data augmentation steps have been implemented as layers within the CNN, this means that they are applied in real time during training allowing training to be more diverse, since for each epoch the same image will be randomly augmented in a different manner. This helps to prevent overfitting by exposing the model to a wider variety of training examples.

2.2 Model Architectures

Three different CNN architectures were implemented with growing complexities. The first model is a simple CNN with one convolutional layer, followed by a max-pooling layer, a flattening layer and two fully connected layers: one with a ReLU activation function and one with a softmax activation function for classification. The second model is a more complex CNN with the first layers being the data augmentation layers, followed by two convolutional layers (each with a max-pooling layer), a flattening layer, and two fully connected layers: one with ReLU activation and one with softmax for classification. The third model is the most complex: the first layer is the data augmentation layer, followed by four convolutional layers (each followed by a max-pooling layer), a flattening layer, and three fully connected layers, two with ReLU activation and one with softmax for classification. After each of the dense layers, a dropout layer with a rate of 0.4 was added to force generalization and prevent overfitting.

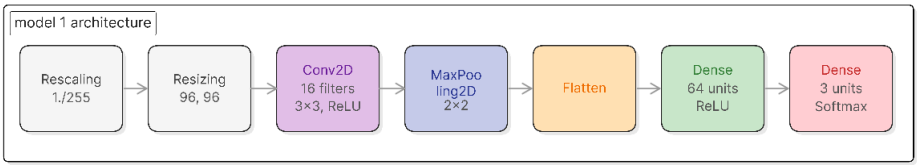


Fig. 2: Architecture of the first CNN model.

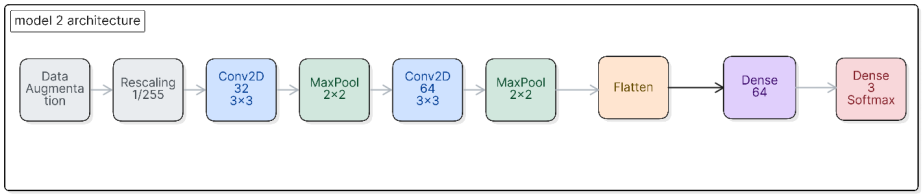


Fig. 3: Architecture of the second CNN model.

The convolutional layers use a 3×3 kernel size in all three models. What changes between the models is the number of filters: 16 filters in the first model, 32 and 64 filters in the second model, and 32, 64, 128, and 256 filters in the third model.

The purpose of convolutional layers is to learn local features from the input images. Increasing the number of filters increases the complexity and representational capacity of the model, allowing it to learn more complex features. However,

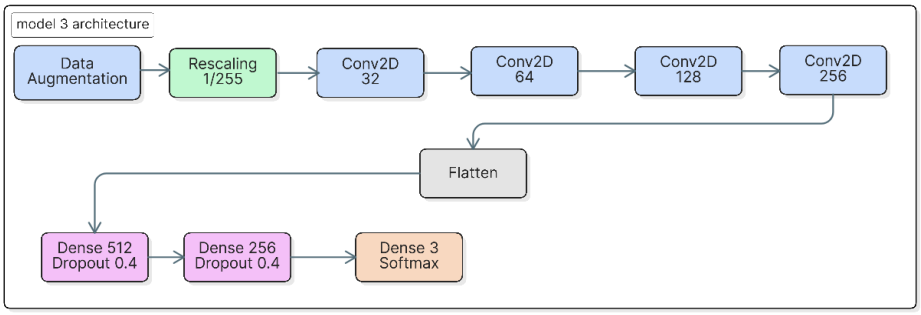


Fig. 4: Architecture of the third CNN model. Maxpooling layers are not shown for simplicity.

a higher number of parameters also increases the risk of overfitting. For this reason, data augmentation and dropout layers are introduced in the second and third models to improve generalization performance.

The max-pooling layers use a 2×2 pool size with a stride of 2. These layers downsample the feature maps produced by the convolutional layers, progressively reducing their spatial dimensions. This process helps the network learn increasingly higher-level features as depth increases. For example, the first convolutional layers typically learn simple patterns such as edges and corners, while deeper convolutional layers learn more complex representations such as shapes, textures, and object-specific patterns.

The fully connected layers perform the final classification based on the features extracted by the convolutional layers. The first fully connected layers use the ReLU activation function, which enables the network to learn non-linear relationships between features. The final fully connected layer uses a softmax activation function, which outputs a probability distribution over the three classes (rock, paper, and scissors) for each input image.

The loss function used for training the models is sparse categorical cross entropy since the target labels are provided as integers (0 for rock, 1 for paper, and 2 for scissors) and the output of the model is a probability distribution over the three classes. The simplified formula for sparse categorical cross entropy that is used in keras is:

$$L(y, \hat{y}) = -\log(\hat{y}_y)$$

Where: - y is the true class label for the input image, represented as an integer (0 for rock, 1 for paper, and 2 for scissors). - $\hat{y}_y$ is the predicted probability for the true class y of the i-th sample. This is obtained from the output of the softmax layer.

Since the training is done using batches of 32 images and the loss is averaged over the batch, the final loss for a batch of N samples can be expressed as:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\log(\hat{y}_{y_i})$$

Where N is the batch size (32 in this case) and y_i is the true class label for the i-th sample in the batch.

The optimizer used for training the models is Adam, which is an adaptive learning rate optimization algorithm that combines the advantages of both momentum and RMSProp.

The first moment represents the exponentially weighted moving average of the gradients:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

Where β_1 is the decay rate for the moving average of the gradients, typically set to 0.9. This means that the current gradient contributes 10% to the first moment, while the previous first moment contributes 90%.

The second moment tracks the exponentially weighted average of the squared gradients:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial L}{\partial w_t} \right)^2$$

Where β_2 is the decay rate for the moving average of the squared gradients, typically set to 0.999.

Since both m_t and v_t are initialized to zero, they are biased toward zero in the early training steps. To efficiently correct for this bias during implementation, the learning rate α is dynamically scaled at each time step t:

$$\alpha_t = \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t}$$

Using this time-step adjusted learning rate along with the raw first and second moments, the final weight update rule is given by:

$$w_{t+1} = w_t - \alpha_t \frac{m_t}{\sqrt{v_t} + \epsilon}$$

where ϵ is a small constant added to avoid division by zero.

While adaptive learning rate is useful, the initial learning rate α is still an important hyperparameter that needs to be tuned for optimal performance. In this project, to find the best learning rate for the most complex model, a grid search was performed over common learning rates (0.003, 0.001, 0.0001, 0.00001) and the best performance was achieved with a learning rate of 0.0001.

Training was performed for a maximum of 15 epochs with a batch size of 32. For model 3, early stopping was implemented to prevent overfitting by monitoring the validation loss during training. If the validation loss does not improve for a specified number of epochs (in this case, 4 epochs), training is stopped early. The learning rate was also reduced in case of a plateau in the validation loss, meaning that if the validation loss does not improve for a certain number of epochs (in the case of model 3, this was 2 epochs), the learning rate is reduced by a factor (0.5) to allow the model to converge to a better minimum.

The metrics used to evaluate the performance of the models during training and testing are accuracy and loss, both on the training and validation sets. When

evaluating the final performance on the test sets (standardized and custom), the metrics used are accuracy, precision, recall, and F1-score. Accuracy measures the overall correctness of the model's predictions, while precision, recall, and F1-score provide more detailed insights into the model's performance for each class, especially in cases where the classes may be imbalanced or when the cost of false positives and false negatives differs, although this is not the case.

The confusion matrix is also used to visualize the performance of the model by showing the number of true positives, true negatives, false positives, and false negatives for each class. This helps to identify what classes the model struggles the most with and proved to be particularly useful while developing the model. The AUC was also computed to compute the area under the receiver operating characteristic (ROC) curve. The AUC provides a single scalar value that summarizes the model's ability to discriminate between classes across all possible classification thresholds. Since this is a multi-class classification problem, the AUC was computed using a one vs rest approach, where the AUC is calculated for each class against all the other classes. Both micro-average (unweighted average) and macro-average AUC (weighted average) were computed to provide a comprehensive evaluation of the model's performance across all classes.

3 Results and Discussion

3.1 Training Performance

Model	Training Accuracy	Validation Accuracy	Training Loss	Validation Loss
Model 1	0.960	0.919	0.163	0.245
Model 2	0.931	0.950	0.207	0.173
Model 3	0.948	0.984	0.168	0.056

Table 1: Training and validation performance for the three models.

In this section training results for the three models are presented and discussed. The performance of the models is evaluated based on the training and validation accuracy and loss, as well as the training curves that show how these metrics evolve over epochs. All values presented in Table 1 are the ones obtained at the end of training.

Model 1: The first model achieved a training accuracy of 96.02% and a validation accuracy of 91.9%. This suggests that the model may be overfitting, this statement is also supported by the two loss values, showing a significantly lower loss for training compared to validation. This becomes evident when looking at the learning curves of the model, where training accuracy is consistently higher than validation accuracy, especially in the later epochs.

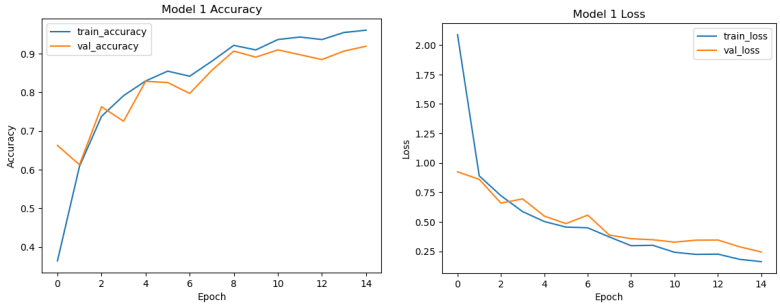


Fig. 5: Training and validation accuracy and loss curves for Model 1.

Model 2: The second model achieved a training accuracy of 93.1% and a validation accuracy of 95%, presenting a major performance improvement compared to model 1. A higher number of parameters does increase the risk of overfitting but the performance of the model suggests otherwise, this is because the data augmentation techniques applied helped to reach a higher level of generalization. The validation accuracy (Figure 6) shows an oscillating behaviour in early epochs, this could be due to the starting learning rate selected being too high, although this problem seems to become less relevant in the later epochs.

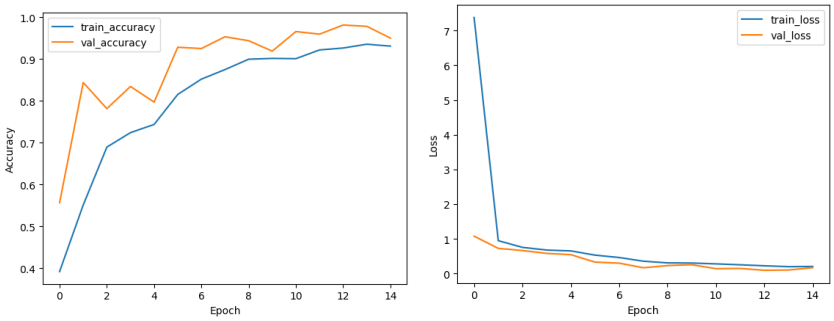


Fig. 6: Training and validation accuracy and loss curves for Model 2.

Model 3: The third model achieved a training accuracy of 94.8% and a validation accuracy of 98.4%. Having the highest number of parameters also means encountering the most risk of overfitting but the introduction of dropout layers that exclude some of the neurons from the weight update process allow the network to generalize better during validation. The grid search for the best starting learning rate proved to be effective; the training curves show how validation performance was consistently better than training performance, with little to no oscillations.

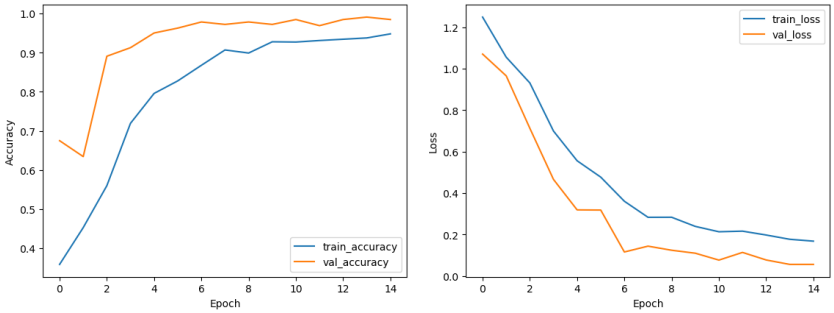


Fig. 7: Training and validation accuracy and loss curves for Model 3.

3.2 Testing performance on the standardized test set

Model	Precision	Recall	Accuracy	F1-score	Micro-AUC	Macro-AUC
Model 1	0.907	0.905	0.905	0.905	0.987	0.986
Model 2	0.956	0.953	0.955	0.954	0.9895	0.989
Model 3	0.976	0.975	0.976	0.975	0.995	0.993

Table 2: Testing performance for the three models.

Model 1: Testing accuracy is similar to validation accuracy, this is expected since both testing and validation sets come from the same original dataset and are originated using a random seed. Overall performance seems to be balanced as shown in the other metrics in Table 2. The one-vs-all AUC shows how the model's performance at identifying certain classes is slightly better than others, the model seems to be more capable at identifying rock samples while struggling more with scissors and paper classes. This behaviour is also shown in the confusion matrix, where most of the false positives belong to either paper or scissors almost equally (Figure 8). This is possibly due to similar features found in the extended fingers, although this is just an assumption.

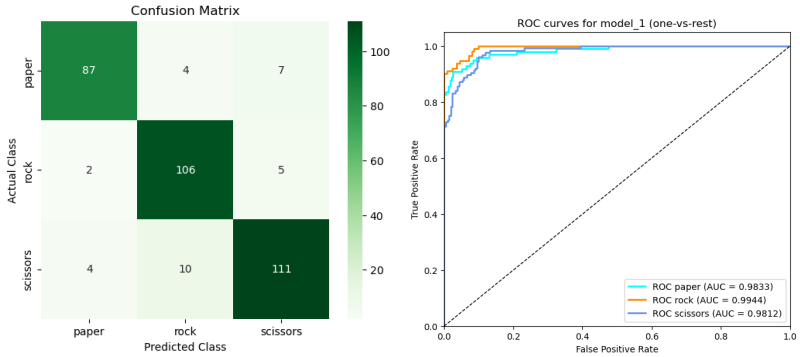


Fig. 8: Confusion matrix for model 1

Model 2: The second model showed significant improvements in performance while maintaining balance (Table 2). The AUC values show the model's ability to differentiate between classes is significantly improved compared to model 1. Looking closely at the ROC curves the paper class is the one that seems to be the hardest to identify by the model. The confusion matrix (Figure 9) shows that most of the misclassifications appear in the paper class being misclassified as scissors, following a similar pattern to the one found in model 1.

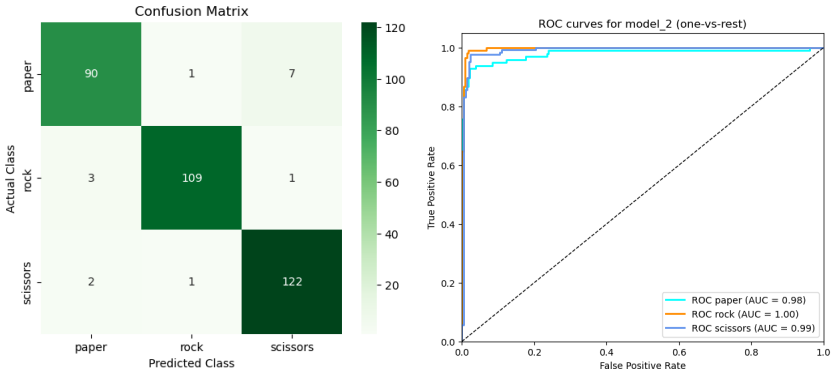


Fig. 9: Confusion matrix for model 2

Model 3: The third model achieved the best performance on the standardized test set (Figure 2). The AUC values show that the model is able to differentiate between classes with a very high degree of accuracy. The trend observed in the previous two models is still present, with misclassifications appearing only in the paper and scissors classes (Figure 10). Error analysis confirmed the assumption

made for the previous models: the misclassified images contain finger positions that are frequent in other classes. Looking at Figure 11, the first image shows a scissors sample misclassified as paper; the fingers appear to be parallel to each other, similar to many samples in the paper class. The second image shows a paper sample misclassified as scissors; the fingers are spread wide apart from each other, which is more common in the scissors class within the dataset. The third image shows a paper sample misclassified as rock, the fingers are close together but the most evident feature is the position within the frame, most of the hand is cut off from the image, leaving only half visible. This blocky shape can be most frequently found in the rock class.

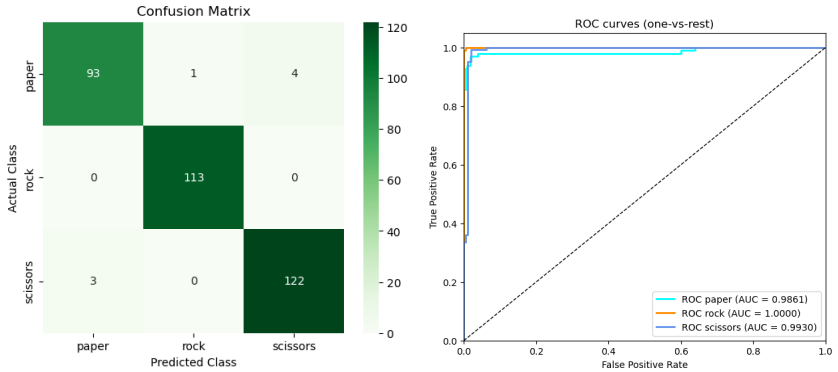


Fig. 10: Confusion matrix for model 3

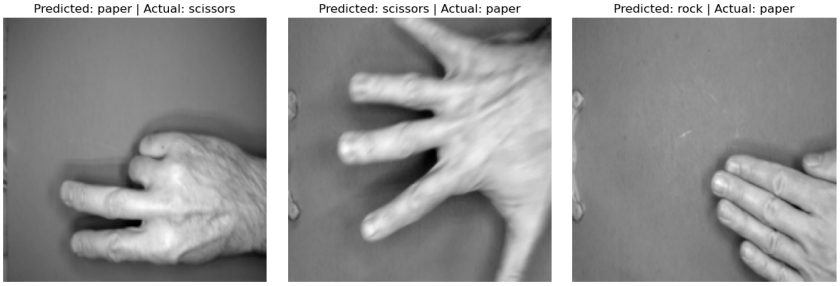


Fig. 11: Misclassified samples by model 3

3.3 Testing performance on custom datasets

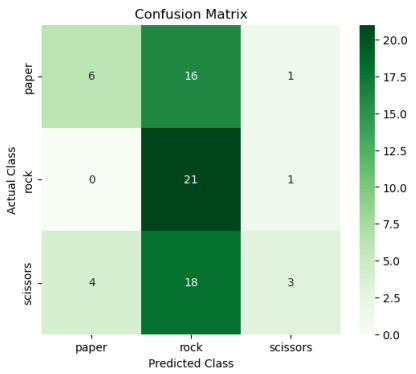
Table 3: Testing performance for the three models on homogeneous backgrounds with frequent rotations.

Model	Precision	Recall	Accuracy	F1-score
Model 1	0.527	0.445	0.429	0.3697
Model 2	0.786	0.7869	0.786	0.786
Model 3	0.879	0.862	0.857	0.858

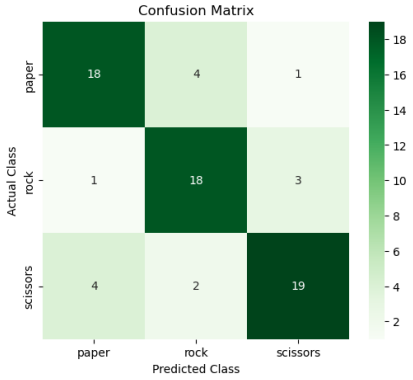
Custom dataset 1: homogeneous backgrounds with frequent rotations

Model 1: The first model showed significant performance degradation compared to the standardized dataset, all performance metrics are significantly lowered such as the accuracy reaching only 42.9% (Table 3). The vast majority of the predictions are within the rock class. This is likely due to the fact that the images in the custom dataset have more variability in terms of lighting and orientation, which makes it harder for the simpler model to generalize.

Model 2: The second model showed a significant improvement in performance compared to the first model, reaching an accuracy of 78.6% (Table 3). The performance was still subject to significant degradation compared to the standardized test set but the model demonstrated a good ability to generalize although this is still a controlled environment, since background conditions are similar to those in the training data.



(a) Confusion matrix for model 1 on the first custom dataset.



(b) Confusion matrix for model 2 on the first custom dataset.

Model 3: The third model achieved the best performance on the first custom dataset, reaching an accuracy of 85.7% showing a significant improvement compared to the other two models and showing a good generalization ability. Comparing the last two models, the performance improvement is better compared to the one obtained in the standardized test set. This suggests that the standardized testing dataset may be too similar to the training set, which could lead to an overestimation of the model's performance.

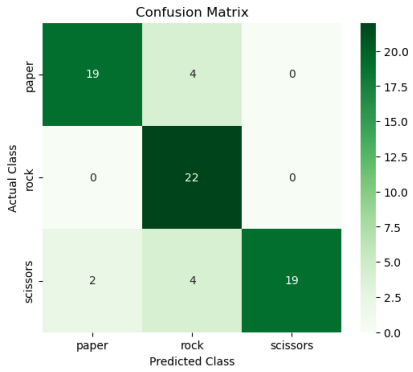


Fig. 13: Confusion matrix for model 3 on the first custom dataset.

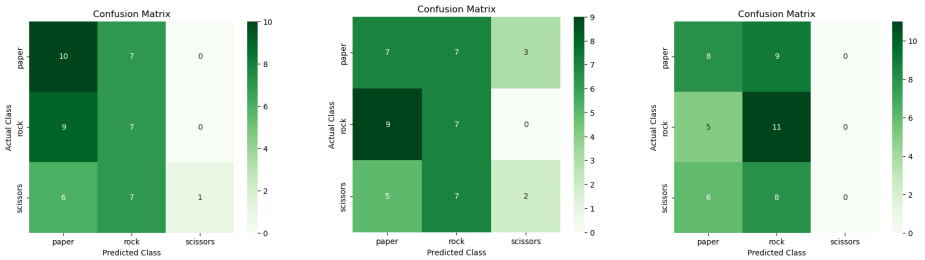
When looking at the confusion matrices for the three models, it can be observed that the trend is still present in the misclassifications, where paper and scissors are the most misclassified classes, likely due to the reliance of the model on finger position as a key feature to classify samples.

Table 4: Testing performance for custom dataset2.

Model	**Precision**	**Recall**	**Accuracy**	**F1-score**
Model 1	0.578	0.366	0.383	0.329
Model 2	0.356	0.331	0.340	0.319
Model 3	0.271	0.386	0.404	0.315

Custom dataset 2: heterogeneous backgrounds with different camera angles When feeding the models images with samples containing different, complex backgrounds along with different camera angles, the performance of all three models suffers a significant degradation reaching performance levels similar to a random classifier (Table 4). This is likely due to the characteristics of the custom dataset samples presenting numerous patterns in the background that are not present in the training data, this makes it harder for the model to focus

and identify relevant features. The confusion matrices show that samples are mostly classified as rock or paper, while scissors is rarely predicted. A possible explanation is that the complex backgrounds containing numerous lines and repetitive patterns may introduce noise in the feature extraction process, resulting in identification of many finger like features or reduce the visibility of those same features.



(a) Model 1 (b) Model 2 (c) Model 3

Fig. 14: Confusion matrices for the three models.

4 Discussion

The results obtained from testing the three CNN models lead to many observations regarding the capabilities and limitations of the models and the dataset used for training. The performance of the models on the standardized dataset was very good, but the significant performance degradation on the generalization setssuggests that, while not actually overfitting during training, the very homogeneous characteristics of the data made both validation and testing performance misleading, leading to overfitting not on the training set but on the dataset as a whole.

The dataset also presented some problems: the background was homogeneous, and almost every sample had the same hand orientation. This problem could be fixed by using stronger data augmentation, randomizing backgrounds, or applying segmentation of the hand before feeding it to the classifier, making it background-invariant.

One further consideration regarding the dataset concerns data labeling. While this is not necessarily the case, since the task of identifying hand gestures is rather simple, it would be useful to assign attributes to the samples identifying the individual they belong to, in order to avoid having the same person appear in both training and validation/testing sets. Otherwise, the model may learn spurious features such as scars or hyperpigmentation rather than the relevant features.

The conversion to grayscale of the images reduced dimensionality but also introduced additional problems which, in hindsight, could have been avoided by using color. First, the introduction of grayscale causes edges to appear sharper, resulting in more evident features than they are in reality, this could lead to misclassifications. Another problem is background noise: when working with homogeneous backgrounds, grayscale performs well, but when introducing heterogeneous backgrounds it becomes harder to distinguish the background from the hand, since color information has been lost. That said, implementing the CNNs using color means that the model might also not work well, considering the homogenous dataset utilized in this project, since the model might also struggle with homogeneous backgrounds of a color different than the training data. Data augmentation could still be applied but some performance degradation is expected. Of course this is just speculation since no testing has been done using all 3 rgb channels.

5 Conclusion

The three CNN models developed in this project followed a principle of increasing complexity, the performance of the models proved to be very good on the main dataset used for training, validating and and testing. The performance on the two generalization sets, representing two levels of increasing difficulty, proved to be adequate for the simpler dataset and similar to that of a random classifier for the second. This suggests that the usage of these models, if ever used outside of this work, should at least fullfill the condition of providing an homogeneous background. The data shows that the homogeneous nature of the dataset caused the three CNNs to overfit on the whole dataset, while maintaining good generalization performance within it.

References

1. Author, Julien de la Bruère-Terreault: Rock Paper Scissors Dataset. Kaggle, https://www.kaggle.com/datasets/drgfreeman/rockpaperscissors/data (2023)

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.